

面向对象编程 两种编程思想对比 ·面向过程：关注完成需求的具体步骤，强调一步步手动实现，适合小型简单需求，核心逻辑是「我要一步步怎么做」 ·面向对象（OOP）：关注具备对应功能的实体，直接调用对象已有的能力完成需求，不需要自己实现所有步骤，大型项目可维护性更强，具备封装、继承、多态三大核心特征 ✨ 金句：“面向过程就是我该做什么，我要做什么；面向对象就是我想让谁来做，这就是思想的转变。”

类与对象的关系 ·类：是创建对象的模板，对一群拥有相同属性（特性）和行为（能力）的事物的抽象描述，不指向具体个体 ·对象：是类的具体实例，描述一个具体的个体 ·关系类比：类=月饼模具，对象=模具压出来的月饼，必须先定义类，才能创建对象

基础语法规则 定义类：class 类名: 类名遵循大驼峰命名规范，类内暂时无内容可先用pass占位 创建对象：变量 = 类名() 这个过程也叫「实例化对象」 操作属性：新增/修改属性用对象.属性名 = 值，获取属性用对象.属性名 调用方法：对象.方法名() 类中定义的方法必须通过对象调用，不能直接调用

```
In [9]: # Python: 数据和方法打包在一起
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age
    def print_info(self): # self 自动指向当前实例
        print(self.name)
    def print_age(self):
        print(self.age)
student = Student("Alice", 20)
student.print_info() # 输出 "Alice"
student.print_age() # 输出 "20"
```

Alice
20

1.__init__初始化方法 调用时机：创建对象时会被Python自动调用，不需要开发者手动调用 核心作用：在对象创建时声明对象属性、设置初始值 语法要求：必须用self.属性名 = 形参的格式声明属性，直接写属性名=值会被识别为仅函数内部有效的局部变量，无法被对象外部访问 2.参数规则：__init__定义了n个除self外的形参，创建对象时必须传入n个实参，参数数量不匹配会报错 属性修改：对象创建后，直接用对象.属性名 = 新值就可以修改属性值，不需要修改类定义 实例化内存特性 同一个类可以实例化出多个对象，每次实例化都会生成新对象，占用不同的内存空间，哪怕两个对象属性完全一致，也是互相独立的个体，属性互不影响。

```
In [ ]: # 规范：类名首字母建议大写 (Animal 而非 animal)
class Animal:
    # 【修复1】必须是双下划线 __init__，这是Python固定的构造方法名
    def __init__(self, name1, type1, age1):
        self.name = name1 # 将传入的参数绑定为实例属性
        self.type = type1
        self.age = age1

    def say(self, info):
        # f-string 格式化输出实例属性和传入的info参数
        print(f"叫{self.name}，是{self.type}，今年{self.age}岁了，{info}")
```

```
# 【修复2 & 3】创建对象和调用方法的代码，必须写在类的外部（全局作用域）
# 且不能缩进到任何方法内部
cat = Animal("小猫", "猫", 2)      # 触发 __init__, 创建cat实例
cat.say("喜欢吃鱼")                # 调用cat的say方法

dog = Animal("小狗", "狗", 3)      # 创建dog实例
dog.say("喜欢吃骨头")              # 调用dog的say方法

# 现在cat和dog是全局变量，可以在这里正常访问其属性
print(cat.name)                    # 输出：小猫
print(dog.type)                    # 输出：狗
```

```
叫小猫 是猫 今年2岁了 喜欢吃鱼
叫小狗 是狗 今年3岁了 喜欢吃骨头
小猫
狗
```

1. **str_魔法函数 概念：**Python中前后带双下划线的方法都属于魔法函数，具备特殊预设功能，__init__、__str__都属于这类 作用：直接打印自定义类的对象时，默认输出看不懂的内存地址信息，定义__str__后，打印对象会自动调用该方法，输出开发者自定义的对象属性信息 使用规则：不能添加自定义参数，必须通过return返回要打印的字符串，可以按需返回部分/全部对象属性，不需要强制返回所有内容
2. 私有属性的规则 定义方法：属性名前加两个下划线_属性名，该属性就变成了私有属性 访问限制：私有属性只能在当前类的内部访问，类外部无法直接访问和修改 外部读取方法：如果需要开放读取权限，可以在类内定义get_属性名()方法，在方法中return私有属性的值，外部调用该方法即可读取，无法直接修改 核心作用：将敏感属性设置为私有，避免被外部随意修改，保证数据安全性
3. 面向对象封装特性 核心逻辑：将属性定义和方法实现都封装在类内部，把复杂的实现细节隐藏在类中，只对外暴露需要开放的访问接口，既保证了数据安全，也提高了代码的可维护性。

```
In [36]: class Friend:
def study(self):
    print("一起复习考研")
def play(self):
    print("一起打瓦")

class HoneyFriend(Friend):
def study1(self):
    print("一起复习考研民法学")
def play(self):
    print("一起打瓦,我来当区")

class Boyfriend(Friend):
def play(self):
    print("一起打瓦,我带你赢")
def cook(self):
    print("给你做饭")

honey = HoneyFriend()
honey.study()    #继承自父类，未重写
honey.study1()   #子类独有的方法，父类没有，调用子类的方法
honey.play()     #重写了父类的方法，调用子类的方法
```

```
boy = Boyfriend()
boy.play()      #重写了父类的方法，调用子类的方法
boy.cook()      #子类独有的方法，父类没有，调用子类的方法
```

```
__一起复习考研
__一起复习考研民法学
__一起打瓦 我来当区
__一起打瓦 我带你赢
给你做饭
```

In [7]: #作业1

```
class Phone:
    def __init__(self,brand,colour,screen_size,price):
        self.brand = brand
        self.colour = colour
        self.screen_size = screen_size
        self.price = price
    def show_info(self,number):
        print(f"品牌: {self.brand}")
        print(f"颜色: {self.colour}")
        print(f"屏幕: {self.screen_size:.1f}英寸")
        print(f"价格: {self.price:,}元")

apple_phone = Phone("apple","白色",6.9,9999)
apple_phone.show_info(2)
```

品牌: apple
颜色: 白色
屏幕: 6.9英寸
价格: 9,999元

In [10]: #作业2

```
class Student:
    def __init__(self,name,age,tel,score,sex):
        self.name = name
        self.age = age
        self.tel = tel
        self.score = score
        self.sex = sex

    def getScore(self):
        print(f"姓名: {self.name}, 成绩: {self.score}")

    def getStudent(self):
        print(f"姓名: {self.name}, 年龄: {self.age}, 电话: {self.tel}, 成绩: {se

students_data = [
    {'name': 'Tom', 'age': 19, 'score': 92, 'sex': '女', 'tel': '15300022839'},
    {'name': 'Jerry', 'age': 20, 'score': 40, 'sex': '男', 'tel': '15300022838'},
    {'name': 'Andy', 'age': 18, 'score': 85, 'sex': '女', 'tel': '15300022837'},
    {'name': 'Jack', 'age': 16, 'score': 65, 'sex': '男', 'tel': '15300022428'},
    {'name': 'Rose', 'age': 17, 'score': 59, 'sex': '男', 'tel': '15300022653'},
    {'name': 'Bob', 'age': 18, 'score': 78, 'sex': '男', 'tel': '15300022867'}
]

# 创建 Student 对象列表
student_list = []
for data in students_data:
    # 按顺序传参: name, age, tel, score, sex
    s = Student(data['name'], data['age'], data['tel'], data['score'], data['sex'])
    student_list.append(s)
```

```
# 打印不及格学生信息 + 统计人数
fail_count = 0
print("=== 不及格学生信息 (成绩 < 60) ===")
for stu in student_list:
    if stu.score < 60:
        stu.getStudent() # 打印全部信息
        fail_count += 1

print(f"\n不及格学生共 {fail_count} 人")
```

=== 不及格学生信息 (成绩 < 60) ===

姓名: Jerry, 年龄: 20, 电话: 15300022838, 成绩: 40, 性别: 男

姓名: Rose, 年龄: 17, 电话: 15300022653, 成绩: 59, 性别: 男

不及格学生共 2 人

```
In [ ]: #作业3
import random

def play_game():
    boy_wins = 0
    girl_wins = 0

    choices = ['石头', '剪刀', '布']

    print("=== 周末去哪儿? 三局两胜决定! ===")
    print("男朋友想看电影 🎬 vs 女朋友想学习 📖\n")

    round_num = 1
    # 主游戏循环
    while boy_wins < 2 and girl_wins < 2:
        print(f"第 {round_num} 局: ")

        while True:
            girl_choice = input("女朋友出 (石头/剪刀/布): ").strip()
            if girl_choice in choices:
                break # 输入合法, 跳出输入循环
            print("输入无效, 请重新输入: 石头、剪刀或布")

        boy_choice = random.choice(choices)
        print(f"男生出: {boy_choice}")
        print(f"女生出: {girl_choice}")

        if boy_choice == girl_choice:
            print("平局! 再来一局\n")
            # 平局不计分, 但轮次需要+1, 所以这里不continue, 让后面的round_num正常
        elif (boy_choice == '石头' and girl_choice == '剪刀') or \
            (boy_choice == '剪刀' and girl_choice == '布') or \
            (boy_choice == '布' and girl_choice == '石头'):
            print("男生赢了这一局! (电影+1)\n")
            boy_wins += 1
        else:
            print("女生赢了这一局! (学习+1)\n")
            girl_wins += 1

        # 每局结束后轮次+1
        round_num += 1
```

```

print("=" * 30)
print(f"最终比分 -> 女朋友(学习): {girl_wins} | 男朋友(电影): {boy_wins}")

if girl_wins == 2:
    print("🏆 获胜方是: 【女朋友】! 今天乖乖去自习室学习吧~ 📖")
else:
    print("🏆 获胜方是: 【男朋友】! 今天开开心心去看电影吧~ 🎬")
print("=" * 30)

if __name__ == "__main__":
    play_game()

```

=== 周末去哪儿? 三局两胜决定! ===

男朋友想看电影 🎬 vs 女朋友想学习 📖

```

第 1 局:
男生出: 剪刀
女生出: 剪刀
平局! 再来一局

第 2 局:
男生出: 布
女生出: 石头
男生赢了这一局: <电影 +1)

```

```

第 3 局:
男生出: 剪刀
女生出: 布
男生赢了这一局: <电影 +1)

```

=====

最终比分 -> 女朋友(学习): 0 | 男朋友(电影): 2

🏆 获胜方是: 【男朋友】! 今天开开心心去看电影吧~ 🎬

=====